

NumPy- Linear Algebra

NumPy package contains **numpy.linalg** module that provides all the functionality required for linear algebra. Some of the important functions in this module are:-

1) **dot()**:- (Dot product of the two arrays)

This function returns the dot product of two arrays. For 2-D vectors, it is the equivalent to matrix multiplication.

```
import numpy.matlib
import numpy as np
a = np.array([[1,2],[3,4]])
b = np.array([[11,12],[13,14]])
np.dot(a,b)
```

2) **vdot()**:- (Dot product of the two vectors)

This function returns the dot product of the two vectors. If the first argument is complex, then its conjugate is used for calculation.

```
import numpy as np
a = np.array([[1,2],[3,4]])
b = np.array([[11,12],[13,14]])
print np.vdot(a,b)
```

3) **inner()**:- (Inner product of the two arrays)

This function returns the inner product of vectors for 1-D arrays. For higher dimensions, it returns the sum product over the last axes.

```
import numpy as np
a= np.array([1,2,3])
b= np.array([0,1,0])
d= np.inner(a,b)
print(d)

import numpy as np
a = np.array([[1,2], [3,4]])
print 'Array a:'
print a
b = np.array([[11, 12], [13, 14]])
print 'Array b:'
print b
print 'Inner product:'
print np.inner(a,b)
```

4) **matmul()**:- (Matrix product of the two arrays)

The **numpy.matmul()** function returns the matrix product of two arrays. While it returns a normal product for 2-D arrays, if dimensions of either argument is >2 , it is treated as a stack of matrices residing in the last two indexes and is broadcast accordingly.

```
import numpy.matlib
import numpy as np
a = [[1,0],[0,1]]
b = [[4,1],[2,2]]
print np.matmul(a,b)
```

```
import numpy.matlib
import numpy as np
a = [[1,0],[0,1]]
b = [1,2]
print np.matmul(a,b)
print np.matmul(b,a)
```

5) **determinant()**:- (Computes the determinant of the array)

Determinant is a very useful value in linear algebra. It is calculated from the diagonal elements of a square matrix. For a 2x2 matrix, it is simply the subtraction of the product of the top left and bottom right element from the product of other two.

In other words, for a matrix $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the determinant is computed as 'ad-bc'. The larger square matrices are considered to be a combination of 2x2 matrices.

The **numpy.linalg.det()** function calculates the determinant of the input matrix.

```
import numpy as np
a = np.array([[1,2], [3,4]])
print np.linalg.det(a)
```

```
import numpy as np
b = np.array([[6,1,1], [4, -2, 5], [2,8,7]])
print b
print np.linalg.det(b)
```

6) **solve()**:-

The **numpy.linalg.solve** function is used to solve a system of linear equations of the form $AX=B$, where A is a matrix and B is a vector or a matrix.

Suppose you have the following system of linear equations:

$$2x+3y=5$$

$$4x+y=6$$

You can represent this system in matrix form $AX=B$ where:

$$A = \begin{bmatrix} 2 & 3 \\ 4 & 1 \end{bmatrix}$$
$$B = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

```
import numpy as np
# Coefficient matrix
A = np.array([[2, 3], [4, 1]])
# Right-hand side vector
B = np.array([5, 6])
# Solve the system of equations
X = np.linalg.solve(A, B)
print("Solution:", X)
```

7) **inv()**:-

We use **numpy.linalg.inv()** function to calculate the inverse of a matrix. The inverse of a matrix is such that if it is multiplied by the original matrix, it results in identity matrix.

```
import numpy as np
x = np.array([[1,2],[3,4]])
y = np.linalg.inv(x)
print x
print y
print np.dot(x,y)
import numpy as np
a = np.array([[1,1,1],[0,2,5],[2,5,-1]])
print ('Array a:')
print (a)
ainv = np.linalg.inv(a)
print ('Inverse of a:')
print (ainv)
print ('Matrix B is:')
b = np.array([[6],[-4],[27]])
print (b)
print ('Compute A-1B:')
x = np.linalg.solve(a,b)
print (x)
# this is the solution to linear equations x = 5, y = 3, z = -2
```

8) **cross()**:- (cross product)

The cross product, also known as the vector product, is an operation on two vectors in three-dimensional space. It results in a new vector that is perpendicular to both of the original vectors. In NumPy, you can compute the cross product of two vectors using the `numpy.cross` function.

```
import numpy as np
# Define the vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
# Compute the cross product
cross_product = np.cross(a, b)
print("Vector a:", a)
print("Vector b:", b)
print("Cross product:", cross_product)
```

Note:- Just like determinant use i,j,k as 1st row

Assignment

1. Write a NumPy program to compute the multiplication of two given matrixes.
2. Write a NumPy program to compute the outer product of two given vectors.
3. Write a NumPy program to compute the cross product of two given vectors
4. Write a NumPy program to compute the determinant of a given square array.
5. Write a NumPy program to evaluate Einstein's summation convention of two given multidimensional arrays.
6. Write a NumPy program to compute the inner product of vectors for 1-D arrays (without complex conjugation) and in higher dimension.
7. Write a NumPy program to compute the Kronecker product of two given multidimension arrays.
8. Write a NumPy program to compute the condition number of a given matrix.
9. Write a NumPy program to find a matrix or vector norm.
10. Write a NumPy program to compute the determinant of an array.
11. Write a NumPy program to compute the inverse of a given matrix.
12. Write a NumPy program to calculate the QR decomposition of a given matrix
13. Write a NumPy program to compute the condition number of a given matrix.
14. Write a NumPy program to compute the sum of the diagonal element of a array.
15. Write a NumPy program to get the lower-triangular L in the Cholesky decomposition of a given array.
16. Write a NumPy program to get the qr factorization of a given array.
17. Write a NumPy program to compute the factor of a given array by Singular Value Decomposition.